TRADEGENIUS

System Architecture

v5.5.0

April 2026

Repo: valira3/stock-spike-monitor
Service: tradegenius.up.railway.app

Contents

1. High-level overview

2. Repo layout

3. Process startup

4. Ticker universe

5. Scheduler & scan loop

- 5.1 Timed jobs (`scheduler_thread`)
- 5.2 Scan loop (`scan_loop`)

6. Trading algorithm — Tiger/Buffalo (v5.0.0, unchanged through v5.1.2)

- 6.1 States
- 6.2 Long protocol — "The Tiger Hunts the Bison"
- 6.3 Short protocol — "The Wounded Buffalo / Gravity Trade"
- 6.4 Sizing — preserved from v4

7. Risk: v5 stop model + portfolio brakes

- 7.1 v5 per-trade stop model
- 7.2 Hard exits (L-P4-R3 / S-P4-R3..R4)
- 7.3 Re-Hunt budget (L-P5-R3 / S-P5-R3)
- 7.4 Portfolio-level brakes — preserved from v4

8. Signal bus & executors

- 8.1 The bus
- 8.2 Executors: TradeGeniusVal & TradeGeniusGene
- 8.3 Live mode flip
- 8.4 Alpaca order idempotency (v5.2.1 H1)
- 8.5 Alpaca client URL hygiene

9. Dashboard

- 9.1 Backend (`dashboard_server.py`)
- 9.2 Index ticker (v4.10 → v4.13.0)
- 9.3 Frontend (`dashboard_static/`)
- 9.4 Shadow strategy P&L (v5.2.0)
- 9.5 Shadow tab (v5.3.0)

10. Environment variables

- 10.1 Required

10.2 Owner whitelist
10.3 Market data
10.4 Persistence (Railway Volume)
10.5 Risk / strategy
10.6 Dashboard
10.7 Executors (Val / Gene)
10.8 Forensic Volume Filter (v5.1.0+)
10.9 Testing
10.10 Do not set

11. Known gotchas

11.1 Dockerfile per-file COPY whitelist
11.2 Yahoo "real-time" is not real-time
11.3 `marketState` is `None`
11.4 Dashboard is read-only
11.5 Don't import `trade_genius` from helper threads
11.6 Executor trade-confirmation DM (v5.0.3 auto-learn)
11.7 Paper and live Alpaca keys are independent (v5.0.4 revert)

12. Persistence

12.1 SQLite store (v5.1.8+)
12.2 Legacy JSON shape (still relevant for the import path)

13. Testing

13.1 Local smoke (`smoke_test.py`)
13.2 Synthetic harness (`synthetic_harness/`)
13.3 CI guards

14. Deployment

14.1 Railway
14.2 Local development
14.3 Logging

15. Maintenance

15.1 Regenerating `trade_genius_algo.pdf`
15.2 Bumping `BOT_VERSION`
15.3 Adding a new top-level Python module
15.4 Reading the live state without restarting

16. v5 strategy quick reference

16.1 State machine
16.2 Rule grid

16.3 Where the v5 logic lives
16.4 Migration from v4
16.5 v5.0.x → v5.5.0 — what changed (and what didn't)

17. Forensic Volume Filter (Anaplan logic) — v5.1.0 → v5.4.1

17.1 Volume Library — parameters
17.2 V-P1 grid (legacy fixed-threshold evaluator)
17.3 Env-driven active config + helper (v5.1.1+)
17.4 `SHADOW_CONFIGS` — fixed analysis configs (v5.1.1 → v5.1.9)
17.4a Bucket-Fill Velocity (v5.1.6)
17.5 Shadow log lines
17.6 Failure modes
17.7 Rollout plan (current)

18. Forensic Capture (v5.1.2)

18.1 Bar archive (`bar_archive.py` — Tier-1.1)
18.2 Per-minute volume log (`[V510-MINUTE]` — Tier-1.2)
18.3 Skipped-candidate log (`[V510-CAND]` — Tier-1.3)
18.4 Entry log enrichment (`[V510-ENTRY]` — Tier-1.4)
18.5 FSM transition log (`[V510-FSM]` — Tier-2.1)
18.6 Pre-trade indicator snapshots (`indicators.py` — Tier-2.2)
18.7 Out of scope (deferred)

19. Long permission gates (G1..G4)

19.1 Four-light gate set (v5.1.0+)

20. Offline backtest CLI (v5.4.0)

20.1 How to run
20.2 What `--validate` does
20.3 CSV ledger columns
20.4 Reused logic

21. Shadow tab charts (v5.4.1)

21.1 Endpoint — `GET /api/shadow_charts`
21.2 Frontend — three chart groups on the Shadow tab
21.3 Polling

22. Nightly backtest (v5.5.0)

22.1 Scheduler entry
22.2 Endpoint — `GET /api/backtest_latest`
22.3 Frontend — "Latest backtest" card
22.4 CLI — `--export-equity`

TradeGenius — System Architecture

> **Version:** v5.5.0 · April 2026 **Repo:** `valira3/stock-spike-monitor` · **Service:** `tradegenius.up.railway.app` **Source of truth:** `STRATEGY.md` (canonical trading-logic spec), `trade_genius.py`, `tiger_buffalo_v5.py`, `volume_profile.py`, `indicators.py`, `bar_archive.py`, `shadow_pnl.py`, `persistence.py`, `dashboard_server.py`, `dashboard_static/{app.js,app.css,index.html}`, `backtest/{loader,ledger,replay,__main__}.py`, `scripts/nightly_backtest.py`

TradeGenius is a Python Telegram-driven trading bot with a paper book, two Alpaca-backed executor mirrors, and a live web dashboard. As of **v5.0.0** it runs the **Tiger/Buffalo two-stage state machine** for both long ("The Tiger Hunts the Bison") and short ("The Wounded Buffalo / Gravity Trade") trades on a small, hand-curated ticker universe; emits ENTRY/EXIT signals on an in-process bus; and (when configured) replicates those signals against Alpaca paper or live via two independent executor bots. **v5.1.x** layered an observation-only **Forensic Volume Filter** (`volume_profile.py`) plus **Forensic Capture** modules (`bar_archive.py` , `indicators.py`) on top of the v5.0.0 algorithm. **v5.1.8** moved persistence from atomic JSON to a SQLite database on the Railway Volume (`persistence.py` , `/data/state.db`). **v5.2.0** added a real-time shadow-strategy P&L tracker (`shadow_pnl.py`); **v5.2.1** hardened Alpaca submits with deterministic `client_order_id` idempotency plus boot-time broker reconcile; **v5.3.0** moved the shadow panel to its own dedicated dashboard tab with click-to-expand per-config detail. **v5.4.0** added the `backtest/` offline replay package and a `python -m backtest.replay` CLI with replay-vs-prod validation. **v5.4.1** layered three Chart.js visualizations onto the Shadow tab (equity curves, day-P&L heatmap, rolling win-rate sparklines) backed by a new `/api/shadow_charts` endpoint with a 30 s server-side cache. **v5.5.0** added a nightly in-process backtest job (22:00 ET) that runs the v5.4.0 replayer against the prior trading day, writes a top-level `/data/backtest_reports/latest.json` index, and surfaces the result as a "Latest backtest" card at the top of the Shadow tab via a new `/api/backtest_latest` endpoint; the CLI also gained an `--export-equity` flag that produces dashboard-shape chart JSON offline. The v5.0.0 Tiger/Buffalo entry/exit decisions are unchanged through v5.5.0 — every v5.1.x / v5.2.x / v5.3.x / v5.4.x / v5.5.0 add is observational, infrastructure, or display-only.

The canonical specification of the trading logic lives in `STRATEGY.md` at the repo root. Sections 6 and 7 below summarize the v5 algorithm and risk model; for any disagreement between this file and `STRATEGY.md` , **`STRATEGY.md` is authoritative**.

App was branded **Stock Spike Monitor** through v3.5.0; renamed to **TradeGenius** in v3.5.1. The pre-v3.5.0 TradersPost mirror portfolio, Robinhood surface, and Gmail/IMAP intake have all been removed and are not part of the current system.

---

1. High-level overview

```
                    ┌──────────────────────────────────────┐
                    │   Railway container (single process)   │
                    │                                        │
   Telegram ──────►│  trade_genius.py                       │
   (main bot,       │   ├─ Scheduler thread                  │
    Val bot,        │   │   • scan_loop() every 60s          │
    Gene bot)       │   │   • timed jobs (09:30/35/55, ...)  │
                    │   │                                     │
                    │   ├─ In-memory state                    │
                    │   │   positions, short_positions,       │
                    │   │   paper_cash, or_high/low, pdc,     │
                    │   │   _trading_halted, etc.             │
                    │   │                                     │
                    │   ├─ Signal bus  ──► Val / Gene ──► Alpaca
                    │   │   _emit_signal(event) async fan-out  │
                    │   │                                     │
                    │   └─ paper_state.py persistence         │
                    │       paper_state.json on Railway Volume │
                    │                                        │
                    │  dashboard_server.py (aiohttp)          │
                    │   ├─ /login, /api/state, /api/indices,  │
                    │   │  /api/executor/{val,gene},          │
                    │   │  /api/errors/{exec}, /stream (SSE)  │
                    │   ├─ /api/version (unauthenticated)      │
                    │   └─ static UI (dashboard_static/)       │
                    └──────────────┬──────────────────────────┘
                                   │
        ┌──────────────────────────┼──────────────────────────┐
        ▼                          ▼                          ▼
   Alpaca Markets          Yahoo v8/chart            Telegram API
   (equity bars,           (cash indices             (commands +
    paper + live            ^GSPC/^IXIC/^DJI/         notifications,
    executors,              ^RUT/^VIX, futures        per-bot tokens)
    Val + Gene)             ES/NQ/YM/RTY)
```

Everything runs in a **single Python process** on Railway. Concurrency model:

- **Main thread** — Telegram async polling (`python-telegram-bot==21.11.1`).
- **`scheduler_thread`** — daemon thread, owns the 60 s scan cadence and the daily timed jobs (`reset_daily_state` , `collect_or` , `eod_close` , …).
- **`dashboard-http`** — daemon thread with its own `asyncio` event loop running an `aiohttp` server on `DASHBOARD_PORT` (default 8080).
- **Per-executor Telegram apps** — Val and Gene each run their own `telegram.ext.Application` event loop (started by `TradeGeniusBase.start()`).
- **Signal listeners** — each `_emit_signal(event)` fans out to listeners in **fresh daemon threads** so a slow Alpaca round-trip on one executor cannot stall the scanner.

2. Repo layout

```
trade_genius.py            # main bot; BOT_VERSION lives here
dashboard_server.py        # aiohttp dashboard backend (~1.9 kLOC)
dashboard_static/
    index.html             # single-page dashboard shell
    app.js                 # two IIFEs: main tab + Val/Gene tab + index ticker
    app.css                # tokens + responsive @media bands
side.py                    # Side enum + SideConfig table (long/short collapse)
paper_state.py             # paper book persistence (extracted in v4.6.0)
error_state.py             # per-executor error rings + dedup gate (v4.11.0)
tiger_buffalo_v5.py        # v5.0.0 Tiger/Buffalo state-machine helpers
volume_profile.py          # v5.1.0 Forensic Volume Filter (shadow only)
indicators.py              # v5.1.2 pure indicator math (rsi/ema/atr/vwap/spread)
bar_archive.py             # v5.1.2 1m bar JSONL persistence (/data/bars/)
persistence.py             # v5.1.8 SQLite-backed state store (/data/state.db)
shadow_pnl.py              # v5.2.0 real-time shadow-strategy P&L tracker
telegram_commands.py       # slash-command handlers
smoke_test.py              # 262 local + 9 prod smoke tests
synthetic_harness/         # 50-scenario byte-equal replay harness
    runner.py, recorder.py, market.py, clock.py, scenarios/, goldens/
scripts/
    build_algo_pdf.py      # regenerates trade_genius_algo.pdf from this file
Dockerfile                 # explicit per-file COPY whitelist (see §11 Gotchas)
requirements.txt
railway.json, nixpacks.toml
.github/workflows/
    version-bump-check.yml # blocks PRs without BOT_VERSION + CHANGELOG bump
    post-deploy-smoke.yml  # runs smoke against tradegenius.up.railway.app
CHANGELOG.md               # full version history
COMMANDS.md                # full Telegram command reference
README.md
trade_genius_algo.pdf      # rendered architecture/algo doc
ARCHITECTURE.md            # this file
```

3. Process startup

`python trade_genius.py` (the `CMD` in the Dockerfile) does the following at module load:

1. Read environment variables (see §10).
2. `_init_tickers()` reads `TICKERS_FILE` (default `tickers.json`); falls back to `TICKERS_DEFAULT` if the file is missing or malformed. `TICKERS_PINNED = ("SPY", "QQQ")` is force-merged in.
3. `_validate_side_config_attrs()` asserts that every `*_attr` field on `SideConfig` (long + short) resolves to a real module-level global. Fail-fast import guard added in v4.9.2 — a renamed dict raises at load time instead of at first entry mid-session.
4. Optional: `dashboard_server.start_in_thread()` if `DASHBOARD_PASSWORD` is set (and ≥ 8 chars).
5. Optional: instantiate `TradeGeniusVal` and `TradeGeniusGene` if their `*_ENABLED` env vars are truthy and paper keys are present; each subscribes to the signal bus via `register_signal_listener`.

6. Start `scheduler_thread` as a daemon.
7. Boot the main Telegram `Application` and call `run_polling()`.

If `SSM_SMOKE_TEST=1` is set the file imports cleanly but skips the network-touching startup (used by `smoke_test.py` and the synthetic harness so they can `import trade_genius` without Telegram credentials).

---

4. Ticker universe

`TICKERS_DEFAULT` is **11 tickers**:

```
AAPL  MSFT  NVDA  TSLA  META  GOOG  AMZN  AVGO  QBTS  SPY  QQQ
```

- **9 spike candidates** — entries fire here.
- **2 pinned filter tickers** — `SPY` and `QQQ` are `TICKERS_PINNED`; they are read on every scan to gate entries by index polarity but are **never opened as positions** and cannot be removed via `/ticker remove`.

Edits go through `add_ticker()` / `remove_ticker()` (Telegram `/ticker add|remove`) which mutate the in-place `TICKERS` list and persist via `_save_tickers_file()` (atomic `.tmp` + `os.replace`). `TRADE_TICKERS` is rebuilt to mirror `TICKERS - TICKERS_PINNED` after every mutation. `TICKERS_MAX = 40` caps the universe so the per-cycle Yahoo budget stays bounded.

Symbol regex: `^[A-Z][A-Z0-9.\-]{0,7}$` (uppercase, US-equity-style; classes, ETFs, and common preferred-stock notations are accepted).

---

5. Scheduler & scan loop

5.1 Timed jobs (`scheduler_thread`)

All times in **America/New_York** (ET); display in the UI is also ET. The trader views the ET clock as the trading clock; an internal display helper renders some Telegram cards in CDT to match the trader's home tz.

Time	Day(s)	Job
09:20	Weekdays	System self-test card
09:30	Weekdays	`reset_daily_state()` — clear OR, daily counts
09:31	Weekdays	System self-test card
09:35	Weekdays	`collect_or()` (in its own thread)
09:36	Weekdays	`send_or_notification()` — OR card to Telegram
15:55	Weekdays	`eod_close()` — force-close all open positions
15:58	Weekdays	`send_eod_report()`
18:00	Sunday	`send_weekly_digest()`

Idempotency: each fire is keyed by `YYYY-MM-DD-HH:MM-day-HH:MM`, kept in a `fired` set, and pruned to today's keys when it grows beyond 200 entries. The fire set is in-memory; a process restart re-fires any job whose minute the new instance lands inside.

5.2 Scan loop (`scan_loop`)

`scheduler_thread` calls `scan_loop()` every `SCAN_INTERVAL = 60` s. The loop's structure:

1. `_refresh_market_mode()` — refresh the OPEN/CHOP/POWER/DEFENSIVE/CLOSED classifier *before* any early-return so the dashboard banner stays correct in after-hours too.
2. Skip if weekend, or before 09:35 ET, or after 15:55 ET (sets `_scan_idle_hours = True` for the dashboard's GATE pill).
3. `_clear_cycle_bar_cache()` — drop the per-cycle 1-minute bar cache.
4. **Regime alert** — re-evaluate `(SPY_cur > SPY_PDC) and (QQQ_cur > QQQ_PDC)`; on transition, fire a Telegram regime card.
5. `manage_positions()` — long stop chain + Red Candle + Lords Left.
6. `manage_short_positions()` — short stop chain + Bull Vacuum + Polarity Shift.
7. `_tiger_hard_eject_check()` — DI/regime hard-eject for open positions.
8. If `_scan_paused` is true (manual `/monitoring pause`), skip new entries; **position management still runs**.
9. For each `t` in `TRADE_TICKERS`: `_update_gate_snapshot(t)` → `check_entry(t)` → `execute_entry(t, px)` → `check_short_entry(t)` → `execute_short_entry(t, px)`.
10. Every exception inside steps 5–9 is wrapped in `report_error(...)` so it shows up in the per-executor health pill (§9.2) and the matching Telegram channel.

`save_paper_state()` runs **every 5 minutes** in its own daemon thread (driven by `last_state_save` in `scheduler_thread`).

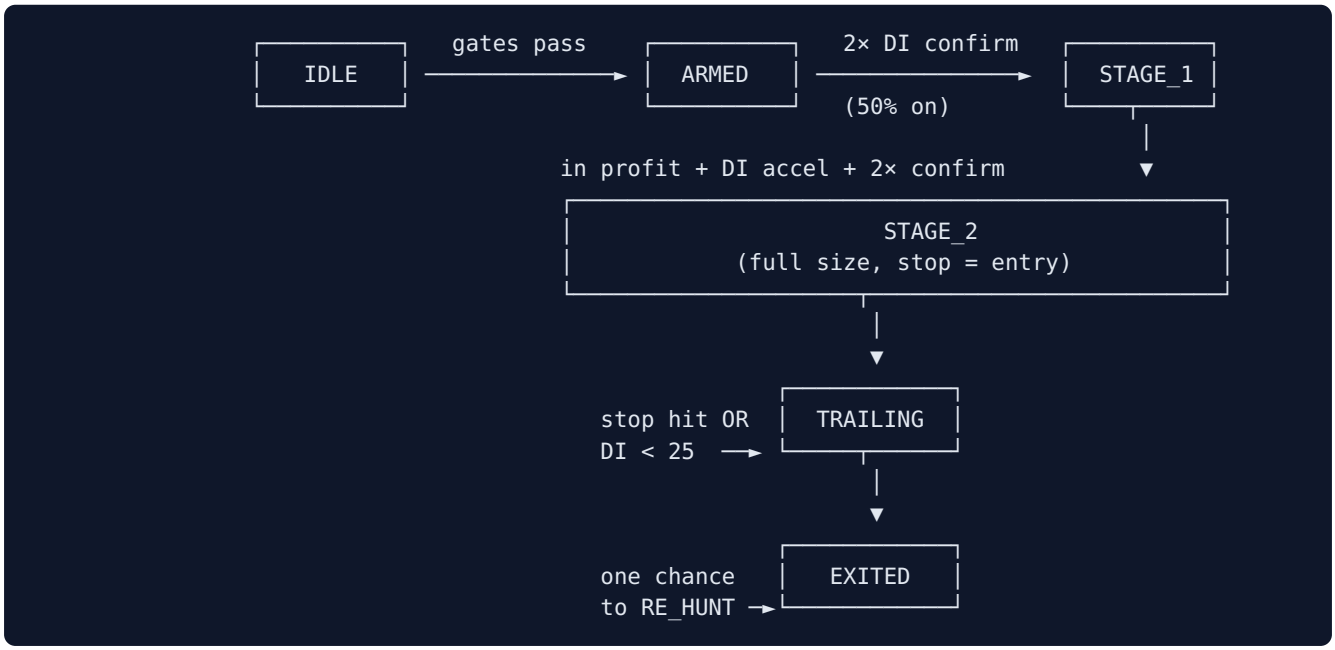
---

6. Trading algorithm — Tiger/Buffalo (v5.0.0, unchanged through v5.1.2)

> **Canonical spec:** `STRATEGY.md` . For any rule disagreement, the spec wins. Every code-level decision in `tiger_buffalo_v5.py` cites a rule ID (e.g. `L-P2-R3`) that maps 1:1 to the spec; smoke-test docstrings reference the same IDs.
>
> **v5.1.x is observation only.** v5.1.0 added the Forensic Volume Filter (§17) and v5.1.2 added the Forensic Capture layer (§18), but **neither one changes any Tiger/Buffalo entry, exit, sizing, or stop decision**. `VOL_GATE_ENFORCE` defaults to `0` and stays at `0` through v5.1.2. The synthetic harness 50/50 byte-equal goldens are still byte-equal under v5.1.2.

v5 reframes both long and short trades as a **two-stage entry, structural-ratchet exit** state machine. Per-ticker per-direction state lives in `trade_genius.v5_long_tracks` / `trade_genius.v5_short_tracks` (persisted in `paper_state.json`).

6.1 States



State enum: `IDLE` , `ARMED` , `STAGE_1` , `STAGE_2` , `TRAILING` , `EXITED` , `RE_HUNT_PENDING` , `LOCKED_FOR_DAY` (helpers in `tiger_buffalo_v5.py`).

6.2 Long protocol — "The Tiger Hunts the Bison"

Phase	Rule IDs	Summary
L-P1	G1..G4	Permission gates: QQQ > QQQ-PDC, SPY > SPY-PDC, ticker > ticker-PDC, ticker > first-hour high (09:30–10:30 ET). All four must be true to leave IDLE.
L-P2	R1..R5	Stage-1 (50% on): `DI+(1m) > 25 AND DI+(5m) > 25` confirmed across **two consecutive closed 1m candles**. Initial stop = low of prior closed 5m candle (hard, does not move during STAGE_1). Record `original_entry_price`.
L-P3	R1..R5	Stage-2 (full size): `DI+(1m) > 30` confirmed across two more consecutive closed 1m candles AND ticker last > original entry ("Winning Rule"). Add remaining 50%. **Safety Lock:** stop on entire 100% position moves to `original_entry_price` ("House Money").
L-P4	R1..R4	TRAILING: every 5m close, ratchet stop UP-ONLY to most recent Higher Low. Hard exits: `ticker.last < current_stop` OR `DI+(1m) < 25` on a closed 1m candle — either flattens 100%.
L-P5	R1..R3	Re-Hunt (one shot): after exit, dormant until ticker > original entry ("reclamation"). On reclamation re-arm with FRESH values. After a second exit → `LOCKED_FOR_DAY`.

6.3 Short protocol — "The Wounded Buffalo / Gravity Trade"

Structural mirror of the long side. The **priority-1 rule (S-P4-R3) inverts** vs. the long side: `DI−(1m) < 25` is checked BEFORE the structural-stop hit. Rationale (per spec): "fear moves faster than greed" — short-side momentum decay typically precedes a squeeze, so the bot covers on DI failure ahead of a stop trigger.

Phase	Rule IDs	Summary
S-P1	G1..G4	Gates: QQQ < PDC, SPY < PDC, ticker < PDC, ticker < opening-range-low (09:30–09:35 ET). If indices are GREEN, shorts are forbidden regardless of ticker weakness.
S-P2	R1..R5	Stage-1 short (50% on): `DI−(1m) > 25 AND DI−(5m) > 25` confirmed × 2. Initial stop = high of prior closed 5m candle.
S-P3	R1..R5	Stage-2: `DI−(1m) > 30` × 2 closed candles AND ticker last < original entry. Add remaining 50%. Safety Lock moves stop to `original_entry_price`.
S-P4	R1..R5	TRAILING: ratchet stop DOWN-ONLY to most recent Lower High on each 5m close. **Priority-1 hard eject:** `DI−(1m) < 25` ⇒ immediate flatten BEFORE structural-stop check. **Priority-2:** `ticker.last > current_stop` ⇒ flatten.
S-P5	R1..R3	Re-Hunt (one shot) on price reclaiming below original entry. Second exit ⇒ `LOCKED_FOR_DAY`.

6.4 Sizing — preserved from v4

v5 changes how the position is **staged in** (50% then 50%), not how the unit is computed.

Parameter	Value
Starting paper capital	$100,000.00 (`PAPER_STARTING_CAPITAL`)
Dollars per entry (paper)	`$10,000` (`PAPER_DOLLARS_PER_ENTRY` env) — this is "100% of unit"
Stage-1 fill size	50% of unit
Stage-2 fill size	remaining 50%
Shares per entry	`paper_shares_for(price)` (unchanged)
Order type	Limit at current market price

Executor sizing is independent and per-executor: `VAL_DOLLARS_PER_ENTRY` and `GENE_DOLLARS_PER_ENTRY` (default $10,000 each). As of **v5.1.4** the live executor `_shares_for(price, ticker=...)` is **equity-aware**: on every entry it calls `client.get_account()` and computes

```
effective_dollars = min(
    dollars_per_entry,
    equity * MAX_PCT_PER_ENTRY/100,
    cash - MIN_RESERVE_CASH,
)
qty = max(1, int(effective_dollars // price))
```

Defaults: `MAX_PCT_PER_ENTRY = 10.0` (i.e. ≤10% of current Alpaca equity per entry) and `MIN_RESERVE_CASH = 500` USD. Worked example: a $30k account with `DOLLARS_PER_ENTRY=10000` will scale each entry down to ~$3,000 and emit `[SIZE_CAPPED]` at INFO. If `effective_dollars < price` the executor returns `qty=0` and logs `[INSUFFICIENT_EQUITY]` instead of submitting an order Alpaca will reject. If `get_account()` itself fails (network blip), the executor logs `[SIZING_FALLBACK]` and falls back to the legacy `int(dollars_per_entry // price)` path so live trading never hard-fails on a transient API error. Paper book sizing (`paper_shares_for` / `PAPER_DOLLARS_PER_ENTRY`) is **unchanged** and remains byte-equal under the synthetic-harness goldens. Per spec C-R7, SPY/QQQ are pinned filter rows on the dashboard and serve as L-P1-G1/G2 / S-P1-G1/G2 inputs; they are NEVER traded directly.

---

7. Risk: v5 stop model + portfolio brakes

> The v4 4-layer stop chain (structural baseline / 0.75% cap / breakeven ratchet / peak-anchored profit-lock ladder) has been **replaced** in v5 with the spec's two-stage stop model below. The v4 ladder math is preserved in source for the synthetic harness baselines but is no longer authoritative.

7.1 v5 per-trade stop model

The stop is a single value per track, evolved through three regimes:

1. **Stage-1 stop (L-P2-R4 / S-P2-R4) — "Emergency Exit."** Set to the low of the prior closed 5m candle (long) or the high of the prior closed 5m candle (short). Hard stop. Does NOT move while the track is in `STAGE_1`.

2. **Stage-2 safety lock (L-P3-R5 / S-P3-R5) — "House Money."** On the Stage-2 fill, the stop on the entire 100% position is moved to `original_entry_price`. The trade is now risk-free vs. its own cost basis.
3. **TRAILING ratchet (L-P4-R1..R2 / S-P4-R1..R2).** On each 5m close after Stage 2, compute the most recent Higher Low (long) or Lower High (short). Ratchet in the favorable direction only — the stop NEVER moves down on a long or up on a short. Implementation: `tiger_buffalo_v5.ratchet_long_higher_low` / `ratchet_short_lower_high`.

7.2 Hard exits (L-P4-R3 / S-P4-R3..R4)

Side	Priority-1 (closed 1m candle)	Priority-2 (every tick)
Long	`DI+(1m) < 25` ⇒ flatten	`ticker.last < current_stop`
Short	`DI−(1m) < 25` ⇒ flatten	`ticker.last > current_stop`

Short side inverts ordering per S-P4-R3: the DI<25 hard eject is priority-1 over the structural-stop check. This is the single most important behavioral asymmetry between the two protocols and is covered by a dedicated smoke test (`v5 S-P4-R3: short DI<25 hard eject fires PRIORITY-1 over structural stop`). Long side: either trigger flattens, order is observability-only.

7.3 Re-Hunt budget (L-P5-R3 / S-P5-R3)

After an exit the track lands in `EXITED` with the re-hunt budget unspent. On reclamation (price climbs back above original entry for longs, below for shorts), the state machine returns to `ARMED` with all stage counters reset and `original_entry_price = None`. After a second exit the track is forced to `LOCKED_FOR_DAY` regardless of subsequent reclamations.

7.4 Portfolio-level brakes — preserved from v4

Spec section C documents the cross-cutting rules; v5 wires each into the existing v4 helpers so the brakes still fire even when the v5 state machine is mid-trade.

Spec	Brake	Implementation
C-R4	Daily-loss-limit (incl. v4.7.0 short-side cap)	`_check_daily_loss_limit()` flips `_trading_halted=True` AND calls `v5_lock_all_tracks("daily_loss_limit")` so every track moves to `LOCKED_FOR_DAY`.
C-R5	EOD force-close at 15:55 ET	`eod_close()` walks `positions` + `short_positions` flattening every open paper position, then calls `v5_lock_all_tracks("eod")` so the next session starts fresh rather than resuming.
C-R6	Sovereign Regime Shield (Eye of the Tiger)	`_sovereign_regime_eject()` — preserved unchanged. When active, all gates are forced false and any open position is flattened.
C-R7	9-ticker spike universe + SPY/QQQ pinned	`TRADE_TICKERS` is the 9-name spike list (excludes SPY/QQQ). `check_breakout` reads SPY/QQQ as polarity inputs only.

`DAILY_LOSS_LIMIT` (default −$500, env-tunable). Once today's realized P&L crosses the floor, `/api/state` exposes `_trading_halted_reason` so the dashboard's GATE pill paints `HALTED`. Both long and short honor the limit (the v4.7.0 fix is preserved).

---

8. Signal bus & executors

8.1 The bus

`_signal_listeners: list[Callable]` plus `_signal_listeners_lock` (a `threading.Lock`). `register_signal_listener(fn)` is idempotent — a re-registration is a no-op so a supervisor restart of an executor cannot double-fire entries against Alpaca.

`_emit_signal(event)` snapshots the listener list under the lock, spawns a fresh daemon thread per listener, and returns immediately. Per-listener exceptions are caught and logged but never propagated.

Event schema:

```
{
    "kind":         "ENTRY_LONG" | "ENTRY_SHORT" | "EXIT_LONG" |
                    "EXIT_SHORT" | "EOD_CLOSE_ALL",
    "ticker":       "AAPL",        # absent on EOD_CLOSE_ALL
    "price":        175.42,        # main's reference price
    "reason":       "BREAKOUT" | "STOP" | "TRAIL" | "RED_CANDLE" |
                    "LORDS_LEFT" | "BULL_VACUUM" | "POLARITY_SHIFT" |
                    "EOD" | "HARD_EJECT_TIGER" | ... ,
    "timestamp_utc":"2026-04-24T13:45:12Z",
    "main_shares":  57,            # audit: shares the paper book traded
}
```

8.2 Executors: TradeGeniusVal & TradeGeniusGene

Both subclass a shared `TradeGeniusBase`. Behavior lives in the base; the subclasses only set `NAME` and `ENV_PREFIX`.

Executor	Name	Env prefix
`TradeGeniusVal`	Val	`VAL_`
`TradeGeniusGene`	Gene	`GENE_`

Per-executor knobs (read at instance construction):

- `*_ALPACA_PAPER_KEY` / `*_ALPACA_PAPER_SECRET`
- `*_ALPACA_LIVE_KEY` / `*_ALPACA_LIVE_SECRET`
- `*_TELEGRAM_TG` (token) / `*_TELEGRAM_CHAT_ID`
- `*_DOLLARS_PER_ENTRY` (default `$10,000`)
- `*_ENABLED` (default on; set to `0` to skip startup)

The owner whitelist is **shared with main** via `TRADEGENIUS_OWNER_IDS` — there are no per-executor `VAL_TELEGRAM_OWNER_IDS` or `GENE_TELEGRAM_OWNER_IDS` (those names appear in `.env.example` for historical readers but the code uses the unified set). Default value: `5165570192,167005578` (Val + Gene).

Routing (clarified in v5.0.3). Each executor's *own* Telegram bot fans trade confirmations out to every learned owner-id → chat-id pair. In other words: Val's trades land on Val's bot, Gene's trades land on Gene's bot, but **every owner sees every executor's confirmations on that executor's bot** because the owner whitelist is shared. Each owner must DM their executor bot once (any message) so the auto-learn hook records the chat_id; the map is persisted at `/data/executor_chats_{name}.json` and survives Railway redeploys (see §11.6). No per-executor `<PREFIX>TELEGRAM_CHAT_ID` env var is required — it is accepted only as a back-compat seed.

Paper/live Alpaca key independence (v5.0.4). Each executor reads its paper credentials from **only** `<PREFIX>ALPACA_PAPER_KEY` / `<PREFIX>ALPACA_PAPER_SECRET` and its live credentials from **only** `<PREFIX>ALPACA_LIVE_KEY` / `<PREFIX>ALPACA_LIVE_SECRET`. There is no fallback from paper to live (or vice versa). v5.0.3 briefly added a fallback from `<PREFIX>ALPACA_PAPER_KEY` to `<PREFIX>ALPACA_KEY` (the un-prefixed name); v5.0.4 reverted it after Val confirmed `GENE_ALPACA_KEY` on Railway is a LIVE key. Operators provisioning a new executor must mint a fresh paper key from a paper Alpaca account and set the paper-prefixed env var — never repurpose un-prefixed keys. See §11.7 for the full incident write-up.

State files (one per mode, per executor):

```
tradegenius_val_paper.json     tradegenius_val_live.json
tradegenius_gene_paper.json    tradegenius_gene_live.json
```

`_load_state()` picks the most-recently-written file so a live restart stays in live; mode flips rebuild the Alpaca client through `_build_alpaca_client(mode=...)`.

8.3 Live mode flip

`set_mode("live", confirm_token)` requires:

1. `confirm_token == "confirm"` (i.e. the operator typed `/mode val live confirm`).
2. `_live_sanity_check()` — build a temp live `TradingClient`, call `get_account()`, verify status contains `ACTIVE`. Logs account number / cash / buying power.

If either gate fails, the mode does not flip and the bot replies with the error reason. Paper flips have no token gate.

8.4 Alpaca order idempotency (v5.2.1 H1)

Every executor submit is routed through `_submit_order_idempotent`, which stamps a deterministic `client_order_id` of the form

```
f"{NAME}-{TICKER}-{utc_iso_minute}-{LONG|SHORT}"
```

(e.g. `Val-AMD-2026-04-24T13:48Z-LONG`) on the request. Alpaca rejects a second submit with the same `client_order_id` as a duplicate; the wrapper catches that rejection, refetches the existing order via `get_order_by_client_order_id`, and returns it as if the original submit succeeded. The net effect is "submit

once, even after a process restart within the same minute." A startup hook, `_reconcile_broker_positions`, walks the live Alpaca positions list at boot, grafts any broker order that is missing from the in-process state back into the matching track, and DMs the executor's owners that a graft happened. Together H1 closes the v5.0–5.2 race where a Railway redeploy mid-minute could have double-submitted an entry.

8.5 Alpaca client URL hygiene

`alpaca-py 0.43.2`'s `RESTClient` builds the final URL as `base_url + "/" + api_version + path`, i.e. it always appends `/v2`. `url_override` therefore must be a **host** (e.g. `https://paper-api.alpaca.markets`) — not a host-with-`/v2`. `_build_alpaca_client` defensively strips trailing `/v2` or `/v2/` so a misconfigured `ALPACA_ENDPOINT_PAPER` or `ALPACA_ENDPOINT_TRADE` env var cannot produce double-prefixed URLs.

---

9. Dashboard

9.1 Backend (`dashboard_server.py`)

`aiohttp` server, runs in a daemon thread with its own asyncio loop.

Endpoint	Auth	Purpose
`GET /`	cookie or login page	Dashboard SPA (`dashboard_static/index.html`)
`POST /login`	rate-limited (5/min/IP)	Form-encoded password; sets HMAC token cookie
`POST /logout`	cookie	Clears `spike_session` cookie
`GET /api/state`	cookie	Aggregate snapshot: portfolio, positions, trades, gates, regime, errors
`GET /api/version`	**none**	Returns `{"version": BOT_VERSION}` — used by the post-deploy poller
`GET /api/indices`	cookie	SPY/QQQ/DIA/IWM/VIX (Alpaca) + ^GSPC/^IXIC/^DJI/^RUT/^VIX (Yahoo) + futures badges
`GET /api/executor/{name}`	cookie	Per-executor tab data; cached 15 s
`GET /api/errors/{executor}`	cookie	Health-pill expanded list (last 10)
`GET /api/trade_log`	cookie	Persistent trade-log tail; supports `limit`, `since`, `portfolio`
`GET /stream`	cookie	Server-Sent Events; pushes a state snapshot every 2 s
`GET /static/*`	none	Served from `dashboard_static/`

Auth: HMAC-SHA256 token stored in `spike_session` cookie. Token = `hex(HMAC(secret, BE-uint64-timestamp)) + ":" + ts`. The secret is generated once on first boot and persisted to `dashboard_secret.key` on the Railway Volume so re-deploys do not invalidate sessions (v3.4.29). Override

via `DASHBOARD_SESSION_SECRET` for tests / forced rotation. Cookie attrs: `HttpOnly; SameSite=Lax; Secure=True; Max-Age=7d`.

`_check_auth()` validates timestamp + HMAC on every authenticated endpoint. POST `/login` enforces a per-IP in-memory rate limit (5 attempts per 60 s window).

State snapshot caching:

- `_cached_snapshot()` — TTL cache of `snapshot()` (~10 s) shared across SSE clients. v4.1.9-dash so concurrent dashboards do not each pay `O(N_positions)` Alpaca round-trips every 2 s.
- `_indices_cache` — 30 s TTL on `_fetch_indices()` (one Yahoo batch per cache miss; ETF prices come from Alpaca via the executor's data client).
- `_executor_cache` — 15 s TTL per `(executor_name)`.

9.2 Index ticker (v4.10 → v4.13.0)

The strip at the top of the dashboard now carries **two stacked sources**:

Alpaca ETF rows (5):

```
SPY  QQQ  DIA  IWM  VIX
```

`_fetch_indices()` calls `client.get_stock_snapshot(StockSnapshotRequest(...))` on the first available executor's paper keys (`_resolve_data_client`). `VIX` is requested separately and tagged with `reason="vix_no_equity_feed"` because Alpaca's equity feed does not carry the volatility index symbol — its row stays as a sentinel placeholder.

Yahoo cash + futures rows (5):

```
^GSPC (S&P 500)   [ES +0.40%]
^IXIC (Nasdaq)    [NQ +0.32%]
^DJI  (Dow)       [YM +0.18%]
^RUT  (Russell 2K)[RTY −0.05%]
^VIX  (VIX)       (no future badge — VX=F is on CFE)
```

`_fetch_yahoo_quote_one(symbol)` hits

```
https://query1.finance.yahoo.com/v8/finance/chart/<URL-ENC(symbol)>?
    interval=1m&range=1d&includePrePost=true
```

with the caret/equals URL-encoded (`^→%5E` , `=→%3D`) so `^GSPC` and `ES=F` both round-trip cleanly. `_fetch_yahoo_quotes(symbols)` fans out across a `ThreadPoolExecutor(max_workers=8)` with a per-request `timeout=6` seconds, so the 9-symbol batch (5 cash + 4 futures) completes in ≈ one request's wall-clock instead of 9 sequential.

The futures badge `[ES +0.40%]` is computed from the future's *own* percent move vs. the future's *own* previous close — not from last-cash vs. future-prev — so the badge tells the trader *where futures are pricing the next session's open*, which is the entire reason to show futures. `^VIX` has no entry in `_YAHOO_INDEX_FUTURE` so its row never paints a badge.

If the Yahoo batch returns **zero** symbols (whole-batch failure), the payload sets `yahoo_ok=false` and `yahoo_error=...` ; the frontend prepends a dim `data delayed` chip while keeping every Alpaca ETF row live ("degrade, don't disappear"). A *partial* miss (e.g. `^RUT` flickering for one cache window) silently omits the missing row.

Session classification — `_classify_session_et()` returns one of `rth | pre | post | closed` from `America/New_York` :

- Weekday `04:00–09:30 ET` → `pre`
- Weekday `09:30–16:00 ET` → `rth`
- Weekday `16:00–20:00 ET` → `post`
- Otherwise (incl. weekends) → `closed`

Used by the frontend AH layer on ETF rows: outside RTH the row paints an extra `AH +0.42 +0.06%` (or `PRE …`) chip in the AH delta's own color (independent of the regular-session change's color). The backend's `_fetch_indices()` returns `marketState=None` from Yahoo's metadata, which is why we classify session locally instead of trusting the upstream field.

9.3 Frontend (`dashboard_static/`)

Single HTML file, two independent IIFEs in `app.js` :

1. **Main dashboard IIFE** (≈ lines 1–797). Owns the `/api/state` poller, `/stream` SSE consumer, KPI rendering, position table, trade table, GATE/Regime/Session pills, `applyGateTriState()` , `__tgApplyHealthPill()` , brand-row clock, LIVE pill, recycle countdown.
2. **Tabs + executors IIFE** (≈ lines 799–1632). Owns the Main/Val/Gene tab switcher, per-executor `/api/executor/{name}` poller (15 s), and the index ticker strip `/api/indices` poller (30 s) including marquee animation.

The two IIFEs communicate via `window.__tgApplyGateTriState` and `window.__tgApplyHealthPill` — explicit named bridges added in v4.10.2 when `Fetch failed` banners surfaced from cross-IIFE access.

LIVE pill (v4.11.5). `#h-tick` always renders `♻ NN` s where `NN` is the seconds to the next scan, or `♻ --` when the backend has no schedule (weekend / scanner idle). The 1 s `streamTickTimer` interval decrements `__nextScanSec` if numeric and unconditionally calls `updateNextScanLabel()` . `#h-tick` is **never** hidden — Val's hard rule preserved across every mobile band.

Index marquee (v4.12.0). After each `renderIndices()` the JS measures `track.scrollWidth` vs `strip.clientWidth` ; on overflow it adds `.idx-marquee` to the strip and **duplicates** the inner items so the CSS `translateX(0 → -50%)` keyframe loops seamlessly. Pause is applied on `:hover` , `:focus-within` , and a tap-to-toggle `.is-paused` class. The whole thing is gated by `prefers-reduced-motion: reduce` which falls back to native `overflow-x: auto` .

Health pill (v4.11.0). Replaced the noisy log-tail card. Tap to expand the last 10 events. Three tiers driven by `error_state.snapshot()` :

- `green` — count == 0
- `warning` (amber) — count > 0 AND no error/critical events today
- `red` — any error or critical event today

Errors fan out to the matching executor's Telegram channel via `report_error()` with a per-`(executor, code)` 5-minute dedup gate (see `error_state.py`). Daily counts reset inside `reset_daily_state()`.

CSS breakpoints (`dashboard_static/app.css`): `@media (max-width: 500/400/380/360 px)` for the mobile cascade; an additional `900 / 640 px` band for sidebar/grid collapse and the index-strip "compact" mode that drops the absolute Δ$ value so 5 items fit on a 390-px iPhone horizontally.

9.4 Shadow strategy P&L (v5.2.0)

`shadow_pnl.py` (`ShadowPnL` class) is the in-process tracker for every shadow config that owns virtual positions. It is paper-portfolio sized (`PAPER_DOLLARS_PER_ENTRY`), marked-to-market on every minute close (`mark_to_market`), and persists rows in the `shadow_positions` table inside `/data/state.db`. The dashboard renders running today + cumulative P&L per config, win rate, and unrealized P&L. v5.2.1 H3 ungated the mark-to-market path from `paper_holds` so a shadow position on a ticker the live paper book is also holding still gets marked.

9.5 Shadow tab (v5.3.0)

Through v5.2.1 the shadow P&L block sat at the bottom of the Main tab. v5.3.0 promotes shadow strategies to a dedicated tab — the dashboard header now carries **four tabs**: Main / Val / Gene / Shadow. The Shadow panel hosts the same per-config running P&L block plus a click-to-expand row per config that calls two new `ShadowPnL` methods:

- `open_positions_for(config_name)` — list of currently open virtual positions for the named config, including the entry timestamp, side, entry price, and live mark.
- `recent_closed_for(config_name, limit=10)` — the last N closed trades for the named config, ordered newest first, with realized P&L per row.

No schema migration ships with v5.3.0 — both methods read the existing `shadow_positions` table that v5.2.0 introduced. The two IIFEs in `app.js` continue to share state through the `window.__tg*` named bridges; the new Shadow tab simply registers a fourth `data-tg-tab` target alongside Main / Val / Gene.

---

10. Environment variables

Dumped from `os.getenv` / `os.environ` in `trade_genius.py` and `dashboard_server.py`.

10.1 Required

Variable	Used by	Notes
`TELEGRAM_TOKEN`	main bot	`@BotFather` token for the main TradeGenius bot
`CHAT_ID`	main bot	Group/channel id (negative for groups)

10.2 Owner whitelist

Variable	Default	Notes
`TRADEGENIUS_OWNER_IDS`	`5165570192`	Comma-separated Telegram user ids; group=-1 TypeHandler
		drops non-owners silently. Renamed from
		`RH_OWNER_USER_IDS` in v3.6.0 — old name no longer read

Production typically sets this to `"5165570192,167005578"` (Val + Gene).

10.3 Market data

Variable	Default	Notes
`FMP_API_KEY`	hard-coded fallback (free key)	PDC + sector / profile lookups

Yahoo Finance is the **sole** source of 1-minute bars from v5.1.3 onward (the prior Finnhub data path was removed in v5.1.3). Alpaca is the order-routing surface and the live websocket source for the v5.1.0 Forensic Volume Filter; FMP is used only for prior-day-close + sector lookups.

10.4 Persistence (Railway Volume)

Variable	Default	Notes
`STATE_DB_PATH`	`/data/state.db`	v5.1.8 SQLite-backed state store (replaces JSON)
`PAPER_STATE_PATH`	`paper_state.json`	Legacy JSON path; kept for one-shot import migration
`PAPER_LOG_PATH`	`investment.log`	Trade log file path (`paper_log()`)
`TRADE_LOG_FILE`	(derived)	Append-only `.jsonl` for full trade log
`TICKERS_FILE`	`tickers.json`	Editable ticker universe

10.5 Risk / strategy

Variable	Default	Notes
`DAILY_LOSS_LIMIT`	`-500`	Halt new entries when realized P&L ≤ floor
`OR_WINDOW_MINUTES`	`5`	Width of the opening range (used for premarket seed)
`OR_STALE_THRESHOLD`	`0.05`	OR vs live drift before entry skip (5%)
`ENTRY_EXTENSION_MAX_PCT`	`1.5`	Max % entry can extend past OR trigger
`ENTRY_STOP_CAP_REJECT`	`1`	Reject entries that would need the 0.75% cap to fire
`TIGER_V2_DI_THRESHOLD`	`25`	DI+/− floor on resampled 5-min bars
`TIGER_V2_REQUIRE_VOL`	`false`	Optional volume confirmation for Tiger v2
`PAPER_DOLLARS_PER_ENTRY`	`10000`	Dollar size per paper entry
`DI_PREMARKET_SEED`	`1`	Include premarket bars in startup DI seed

10.6 Dashboard

Variable	Default	Notes
`DASHBOARD_PASSWORD`	unset	If unset, dashboard does **not** start. Min 8 chars.
`DASHBOARD_PORT`	`8080`	aiohttp listen port
`DASHBOARD_SESSION_SECRET`	unset	Override the persisted `dashboard_secret.key` HMAC key
`DASHBOARD_TRUST_PROXY`	unset	Set to `1` behind Railway / Cloudflare for client-IP

10.7 Executors (Val / Gene)

Each executor uses the prefix below; replace `{P}` with `VAL_` or `GENE_`. Paper and live keys are **strictly independent** — paper mode reads only `<PREFIX>ALPACA_PAPER_KEY` / `<PREFIX>ALPACA_PAPER_SECRET`, live mode reads only `<PREFIX>ALPACA_LIVE_KEY` / `<PREFIX>ALPACA_LIVE_SECRET`. No fallback. (v5.0.4 reverted the brief v5.0.3 paper→un-prefixed fallback; see §11.7.)

Variable	Default	Notes
`{P}ENABLED`	`1`	Set to `0` (or `false`) to skip startup
`{P}ALPACA_PAPER_KEY`	empty	Paper-only Alpaca key. Required to start in paper mode.
`{P}ALPACA_PAPER_SECRET`	empty	Paper-only secret.
`{P}ALPACA_LIVE_KEY`	empty	Live-only key. Required only to flip live.
`{P}ALPACA_LIVE_SECRET`	empty	Live-only secret.
`{P}TELEGRAM_TG`	empty	Per-executor Telegram bot token
`{P}TELEGRAM_CHAT_ID`	empty	v5.0.3: optional seed only (auto-learn fills the map)
`{P}EXECUTOR_CHATS_PATH`	`/data/executor_chats_{name}.json`	v5.0.3 auto-learned chat-map
`{P}DOLLARS_PER_ENTRY`	`10000`	Dollar size per executor entry
`{P}MAX_PCT_PER_ENTRY`	`10.0`	v5.1.4 equity-aware cap: max % of equity per entry
`{P}MIN_RESERVE_CASH`	`500`	v5.1.4 equity-aware cap: USD cash floor after entry
`ALPACA_ENDPOINT_PAPER`	unset	URL host override (no `/v2`); shared by both executors
`ALPACA_ENDPOINT_TRADE`	unset	URL host override (no `/v2`); shared by both executors

Production Railway naming (as of v5.3.0).

- Val executor — paper. `VAL_ALPACA_PAPER_KEY` (26 chars) and `VAL_ALPACA_PAPER_SECRET` (44 chars) are set on Railway from Val's paper Alpaca account. Val executor starts in paper mode at boot.
- Gene executor — currently NOT booting paper. `GENE_ALPACA_KEY` / `GENE_ALPACA_SECRET` exist on Railway but are **LIVE** keys, and the paper-prefixed names (`GENE_ALPACA_PAPER_KEY` / `GENE_ALPACA_PAPER_SECRET`) are unset — so the executor startup gate correctly logs `[Gene] skipped (GENE_ALPACA_PAPER_KEY set=False)` and skips. To bring Gene up in paper mode, mint a fresh paper key from Gene's paper Alpaca account and set the paper-prefixed names; do NOT rename the existing live keys.

10.8 Forensic Volume Filter (v5.1.0+)

Variable	Default	Notes
`VOL_GATE_ENFORCE`	`0`	Master enforcement flag. Stays `0` through v5.3.0 (shadow only).
`VOL_GATE_TICKER_ENABLED`	`1`	Active config: anchor on per-ticker volume gate.
`VOL_GATE_INDEX_ENABLED`	`1`	Active config: anchor on QQQ "Market Tide" gate.
`VOL_GATE_TICKER_PCT`	`70`	Active-config ticker threshold (percent of ToD avg).
`VOL_GATE_QQQ_PCT`	`100`	Active-config QQQ threshold (percent of ToD avg).
`VOL_GATE_INDEX_SYMBOL`	`QQQ`	Hard-locked to QQQ per Val.
`VOLUME_PROFILE_DIR`	`/data/volume_profiles`	Per-ticker baseline JSONs.

Defaults preserve the v5.1.1 Apr 20-24 backtest-recommended thresholds. The five `SHADOW_CONFIGS` (TICKER+QQQ, TICKER_ONLY, QQQ_ONLY, GEMINI_A, BUCKET_FILL_100) plus the two event-driven extras (REHUNT_VOL_CONFIRM, OOMPH_ALERT) are **not** env-driven — they are hard-coded module constants in `volume_profile.py` and `trade_genius.py`; env vars only control which one would gate trades if `VOL_GATE_ENFORCE=1`. See §17 for details.

10.9 Testing

Variable	Default	Notes
`SSM_SMOKE_TEST`	unset	When `=1`, skips the network-touching startup at import time

10.10 Do not set

`TZ` — the bot uses `zoneinfo` and `pytz` internally; setting this breaks scheduling. Documented in `.env.example`.

---

11. Known gotchas

11.1 Dockerfile per-file COPY whitelist

`Dockerfile` does **not** `COPY . .` — it explicitly copies each top-level Python module:

```
COPY trade_genius.py .
COPY telegram_commands.py .
COPY paper_state.py .
COPY side.py .
COPY error_state.py .
COPY tiger_buffalo_v5.py .   # v5.0.0
COPY volume_profile.py .     # v5.1.0
COPY indicators.py .         # v5.1.2
COPY bar_archive.py .        # v5.1.2
COPY persistence.py .        # v5.1.8
COPY shadow_pnl.py .         # v5.2.0
COPY dashboard_server.py .
COPY dashboard_static/ ./dashboard_static/
```

Footgun. Any new top-level Python module must be added to this whitelist in the same PR or the container crashes on boot with `ModuleNotFoundError`. v4.11.0 shipped `error_state.py` without the matching `COPY` line and prod stayed 502 for ~3 hours (`v4.11.1` was the one-line fix). v5.0.0 shipped `tiger_buffalo_v5.py` with the same omission and prod was again 502 until `v5.0.2`. v5.0.2 added a CI infra-guard test (`smoke_test.py: infra: Dockerfile COPY whitelist includes every top-level imported module`) that parses every `import` / `from` line in `trade_genius.py`, intersects against local `.py` modules, and grep-extracts the `COPY <module>.py` directives in the Dockerfile — any missing entry fails CI before merge. v5.1.0 added `COPY volume_profile.py`; v5.1.2 added `COPY indicators.py` and `COPY bar_archive.py`; v5.1.8 added `COPY persistence.py`; v5.2.0 added `COPY shadow_pnl.py` per this guard.

`COPY tickers.json` is intentionally absent — the file lives on the Railway Volume (`TICKERS_FILE=/data/tickers.json`) and is created at runtime by `_save_tickers_file()`.

11.2 Yahoo "real-time" is not real-time

Yahoo's `v8/finance/chart` endpoint returns delayed quotes (typically ~15 minutes for cash indices). The dashboard treats the cash + futures rows as *informational* — the bot never trades off Yahoo data; entries and stop management run off the Alpaca / FMP feed. The frontend's `data delayed` chip is the operator-facing signal that the Yahoo fallback engaged for the *whole batch*.

11.3 `marketState` is `None`

Yahoo's chart-endpoint metadata `marketState` field comes back `null` on most responses regardless of session. The dashboard intentionally does **not** trust it; `_classify_session_et()` derives the session from the local `America/New_York` clock instead.

11.4 Dashboard is read-only

By design, no dashboard endpoint mutates bot state, places orders, or toggles anything. Every shared-state read is best-effort and silent on failure (`_safe(fn, default)`); a dashboard exception cannot tank the bot.

11.5 Don't import `trade_genius` from helper threads

`trade_genius.py` is the program's `__main__`; doing a fresh `import trade_genius` from another thread re-executes the module top level (which calls `loop.add_signal_handler(...)` from a non-main thread and

crashes). `dashboard_server._ssm()` resolves the live module via `sys.modules['__main__']` first; helper modules (`paper_state.py` , `telegram_commands.py`) follow the same pattern.

11.6 Executor trade-confirmation DM (v5.0.3 auto-learn)

Each executor's Telegram bot ships its trade confirmations via `_send_own_telegram` , which fans out to a learned `owner_id -> chat_id` map persisted at `/data/executor_chats_{name}.json` (path overridable via `<PREFIX>EXECUTOR_CHATS_PATH`). The map is populated transparently on the existing `_auth_guard` choke point — any inbound DM from an owner registers that owner's chat_id, atomically writes to disk, and survives Railway redeploys. **No `<PREFIX>TELEGRAM_CHAT_ID` env var is required** (it's accepted as a back-compat seed only). If the map is empty at signal time, the bot logs once at startup (`[Val] notifications EMPTY — DM this executor's bot /start ...`) and silently skips that signal — the trade still hits Alpaca, but the operator must DM the bot once to enable confirmations. This is the bug v5.0.3 fixed; pre-5.0.3 the executor silently dropped every confirmation when `TELEGRAM_CHAT_ID` was unset on Railway.

11.7 Paper and live Alpaca keys are independent (v5.0.4 revert)

Alpaca paper keys and live (real-money) keys are independent credentials issued against different endpoints. They are **not** interchangeable and must **never** be silently substituted for one another. v5.0.3 briefly added a fallback in `TradeGeniusBase.__init__` that read `<PREFIX>ALPACA_PAPER_KEY` and fell back to `<PREFIX>ALPACA_KEY` if the paper key was unset; v5.0.4 reverted that fallback after Val confirmed `GENE_ALPACA_KEY` on Railway is a LIVE key. Had Gene's executor instantiated under v5.0.3, paper-mode orders would have been routed through live credentials. The correct contract — enforced both by `__init__` (v5.0.4 strict reads) and the executor startup gate at module scope — is that paper mode reads **only** `<PREFIX>ALPACA_PAPER_KEY` / `<PREFIX>ALPACA_PAPER_SECRET` , and live mode reads **only** `<PREFIX>ALPACA_LIVE_KEY` / `<PREFIX>ALPACA_LIVE_SECRET` . Operators provisioning a new executor must set the paper-prefixed env vars from a paper Alpaca account; do not repurpose un-prefixed keys, which by repo convention are the live-account credentials.

---

12. Persistence

12.1 SQLite store (v5.1.8+)

`persistence.py` owns the canonical state store at `STATE_DB_PATH` (default `/data/state.db` on the Railway Volume). The legacy `paper_state.json` is read once on first boot if the SQLite file does not yet exist and its rows are imported into the new schema; subsequent boots ignore the JSON. The store holds the same logical fields the JSON held — `paper_cash` , `positions` , `short_positions` , `paper_trades` , `paper_all_trades` , `daily_entry_count` , `daily_short_entry_count` , `or_high` , `or_low` , `pdc` , `_trading_halted` , `_trading_halted_reason` , `bot_version` — plus the v5.2.0 `shadow_positions` table backing `shadow_pnl.ShadowPnL` . Writes are routed through `persistence.save_*` helpers; failures never raise into the trading loop. The store survives Railway redeploys via the persistent `/data` volume and is the source of truth for `_load_state()` after v5.1.8.

12.2 Legacy JSON shape (still relevant for the import path)

The pre-v5.1.8 `paper_state.json` shape is preserved here because the JSON-to-SQLite import path reads it verbatim and the synthetic harness golden files retain the same shape:

```
{
  "paper_cash": 97543.21,
  "positions": {
    "NVDA": {
      "entry_price": 142.50, "shares": 70, "entry_count": 1,
      "stop": 141.44, "initial_stop": 141.23,
      "trail_high": 143.80, "trail_active": true,
      "entry_time": "2026-04-24T13:42:11Z"
    }
  },
  "short_positions": { "...": "..." },
  "paper_trades":     [ "...today's trades..." ],
  "paper_all_trades": [ "...all trades, capped at 500..." ],
  "daily_entry_count":       { "NVDA": 1 },
  "daily_short_entry_count": { "NVDA": 0 },
  "or_high": { "NVDA": 142.10, "...": "..." },
  "or_low":  { "...": "..." },
  "pdc":     { "SPY": 565.30, "QQQ": 480.18, "...": "..." },
  "_trading_halted": false,
  "_trading_halted_reason": "",
  "bot_version": "4.13.0"
}
```

A separate append-only `trade_log.jsonl` (path resolved by `TRADE_LOG_FILE`) logs every entry/exit with full context — the dashboard's `/api/trade_log` reads its tail.

Unknown fields on disk (e.g. legacy `avwap_data`, `avwap_last_ts`) are silently ignored on load so old state files survive code purges.

---

13. Testing

13.1 Local smoke (`smoke_test.py`)

262/262 tests at v5.3.0 (was 161 at v5.0.4, 220 at v5.1.2, 262 at v5.3.0). Tests are grouped via the `@t(name)` decorator. Runs on every PR via `.github/workflows/post-deploy-smoke.yml` (after the prod rollout poll succeeds). Coverage spans the trade-decision arithmetic (stop-chain layers, breakeven ratchet, ladder tiers, entry-extension guard), data helpers (`_classify_session_et`, `_or_price_sane`), state I/O (now `persistence.py` / SQLite via v5.1.8), the index payload schema, the Yahoo helper API surface, the v5 Tiger/Buffalo rule grid (every `L-P*-R*` / `S-P*-R*` / `C-R*` rule has at least one test docstring citing the rule ID), the v5.0.2 Dockerfile COPY infra-guard, the v5.0.3 chat-map auto-learn / fan-out, the v5.0.4 paper-key strict read, the v5.1.0 `[VOLPROFILE]` section (calendar helpers, `session_bucket` boundaries, `evaluate_g4` PASS/BLOCK paths, disable-on-30-symbol cap, JSON round-trip), the v5.1.1 env-driven A/B toggles + parallel `SHADOW_CONFIGS` emit, the v5.1.2 forensic capture additions (`bar_archive.write_bar` + `cleanup_old_dirs`, `indicators.rsi14` / `ema9` / `ema21` / `atr14` / `vwap_dist_pct` / `spread_bps` happy-

path + insufficient-bars-returns-None, emitter formats for `[V510-MINUTE]` / `[V510-CAND]` / `[V510-FSM]` / `[V510-ENTRY]`), the v5.1.4 equity-aware sizing caps (`MAX_PCT_PER_ENTRY`, `MIN_RESERVE_CASH`), the v5.1.6 bucket-fill velocity streams (`[V510-VEL]` / `[V510-IDX]` / `[V510-DI]`), the v5.1.8 SQLite migration (JSON-import idempotency, schema, write barriers), the v5.1.9 `REHUNT_VOL_CONFIRM` arm/check + `OOMPH_ALERT` minute-1/2 state machine, the v5.2.0 `shadow_pnl` tracker (open/close/MTM, `open_positions_for`, `recent_closed_for`), the v5.2.1 idempotency layer (`_submit_order_idempotent` deterministic `client_order_id`, `_reconcile_broker_positions` startup graft, EOD orphan force-close at entry_price with `EOD_NO_MARK`, shadow MTM ungated from `paper_holds`, `_v521_all_shadow_config_names` registry, `(ticker, side)`-keyed REHUNT watch), and the v5.3.0 dashboard wiring (per-config open + recent-closed list, Shadow tab visibility), plus dashboard-route contract tests against a stub aiohttp app.

13.2 Synthetic harness (`synthetic_harness/`)

50 named, byte-equal-replay scenarios under `synthetic_harness/scenarios/{long_entries, short_entries, long_closes, short_closes, scan_loops, edge_cases}.py` with goldens under `synthetic_harness/goldens/*.json`. CLI:

```
python -m synthetic_harness list
python -m synthetic_harness record <scenario>
python -m synthetic_harness replay <scenario>
python -m synthetic_harness diff   <scenario>
```

Each scenario seeds module state through `_reset_module_state`, drives the bot through a `FrozenClock` and `SyntheticMarket`, and records `OutputRecorder` events. `replay_scenario()` strips `trade_genius_version` from both the observed and golden dicts before compare (v4.11.5), so a `BOT_VERSION` bump alone never invalidates all 50 goldens. `record_scenario()` still stamps the current version into freshly-recorded goldens.

`smoke_test.py --synthetic` lifts the local smoke total by replaying all 50 goldens. **v5.1.0 / v5.1.1 / v5.1.2 are observation-only**, so the 50/50 byte-equal still passes after each release without re-record.

13.3 CI guards

- **`version-bump-check.yml`** — blocks PRs to `main` unless **both** `BOT_VERSION` in `trade_genius.py` and the top `##` heading in `CHANGELOG.md` change in the diff. **Escape hatch:** include the literal token `[skip-version]` anywhere in the PR title or body. Use it only for docs-only / CI-only changes.
- **`post-deploy-smoke.yml`** — fires on every push to `main`. Polls `/api/version` against `https://tradegenius.up.railway.app` until the new `BOT_VERSION` is live (or times out at 5 min), then runs the full smoke suite. v4.11.4 repointed `DASHBOARD_URL` here from the pre-rename Railway domain that had been 404'ing since v3.5.1.

---

14. Deployment

14.1 Railway

1. Connect the GitHub repo. Railway auto-builds and deploys on every push to `main` (`railway.json` + `nixpacks.toml` ; Docker builder is the canonical path via `Dockerfile`).
2. Attach a Railway Volume; mount it at `/data` .
3. Set the env vars listed in §10. At minimum: `TELEGRAM_TOKEN` , `CHAT_ID` , `TRADEGENIUS_OWNER_IDS` , `DASHBOARD_PASSWORD` , plus the executor key sets you want active.
4. Verify rollout via `https://tradegenius.up.railway.app/api/version` .

14.2 Local development

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env  # then edit
DASHBOARD_PASSWORD=devpassword python trade_genius.py
```

Smoke tests against the running local instance:

```
DASHBOARD_URL=http://localhost:8080 \
DASHBOARD_PASSWORD=devpassword \
python smoke_test.py --local --prod
```

14.3 Logging

Dual handler: file (`trade_genius.log`) + stdout. All stdout/stderr is visible in the Railway dashboard. Errors that go through `report_error()` also fan out to the matching executor's Telegram channel (with a 5-minute per-`(executor, code)` dedup) and increment the dashboard health pill's per-executor count.

---

15. Maintenance

15.1 Regenerating `trade_genius_algo.pdf`

```
pip install weasyprint
python scripts/build_algo_pdf.py
```

The script renders this `ARCHITECTURE.md` plus a cover page and table-of-contents into `trade_genius_algo.pdf` at the repo root. Re-run it after any architecture change you want reflected in the PDF; commit the regenerated binary in the same PR. The script is intentionally a single file with no project-specific dependencies beyond `weasyprint` and the markdown source so it survives module renames.

15.2 Bumping `BOT_VERSION`

Bump the constant on line 61 of `trade_genius.py` AND prepend a `## v{N} — YYYY-MM-DD — <one-line summary>` heading at the top of `CHANGELOG.md`. The CI `version-bump-required` job blocks the PR otherwise. For **docs-only** or **CI-only** PRs (this one included), leave `BOT_VERSION` alone and put `[skip-version]` in the PR title.

15.3 Adding a new top-level Python module

Always add a matching `COPY <name>.py .` line to the `Dockerfile`. The container will not pick the file up otherwise and will crash on boot. (See §11.1.)

15.4 Reading the live state without restarting

Every piece of bot state is reachable through the dashboard's `/api/state`, `/api/executor/{name}`, `/api/errors/{executor}`, and `/api/trade_log` endpoints — none of them mutate, all of them are cookie-authenticated, and the state snapshot is cached for ≤ 10 s so hammering the dashboard does not stall the scanner.

---

16. v5 strategy quick reference

> Mirror summary of `STRATEGY.md` for at-a-glance reading. The spec is still authoritative — this section just collects the rule-ID grid in one place so a reader paging through the architecture doc can find the entry/exit thresholds without flipping files.

16.1 State machine

```
IDLE —[L-P1/S-P1 gates]→ ARMED —[L-P2-R2 / S-P2-R2 2× DI≥25]→ STAGE_1
   STAGE_1 —[L-P3-R2 / S-P3-R2 2× DI≥30 + winning rule]——→ STAGE_2
   STAGE_2 —[next 5m close]——→ TRAILING
   TRAILING —[stop / DI<25]——→ EXITED —[reclaim once]→ ARMED
   EXITED —[2nd exit]→ LOCKED_FOR_DAY
```

16.2 Rule grid

ID	Where it fires
L-P1-G1..G4	Long permission gates (4 lights on dashboard).
L-P2-R1..R5	Stage-1 long entry (50%, prior-5m-low stop, record entry).
L-P3-R1..R5	Stage-2 long add (full size, safety lock to entry price).
L-P4-R1..R4	TRAILING long: 5m HL ratchet up; stop OR DI<25 ⇒ flatten.
L-P5-R1..R3	Long re-hunt: dormant until reclaim; one shot; 2nd exit ⇒ LOCK.
S-P1-G1..G4	Short permission gates; indices-green vetoes shorts entirely.
S-P2-R1..R5	Stage-1 short entry (50%, prior-5m-high stop).
S-P3-R1..R5	Stage-2 short add; safety lock to entry.
S-P4-R1..R5	TRAILING short: 5m LH ratchet down; **DI<25 priority-1.**
S-P5-R1..R3	Short re-hunt; 2nd exit ⇒ LOCK.
C-R1	Long ⊥ short on same ticker per session.
C-R2	DMI period = 14 (down from v4's 15).
C-R3	Closed-candle entries; tick-rate exits.
C-R4	Daily-loss-limit ⇒ LOCKED_FOR_DAY for all tracks.
C-R5	EOD 15:55 ET force-close.
C-R6	Sovereign Regime Shield global kill.
C-R7	9-name spike universe; SPY/QQQ pinned, never traded.

16.3 Where the v5 logic lives

Concern	File / function
Rule logic (pure helpers)	`tiger_buffalo_v5.py` (no imports from `trade_genius`)
Per-track state	`trade_genius.v5_long_tracks` / `v5_short_tracks`
Mutex + active direction	`trade_genius.v5_active_direction`
DMI on 1m + 5m timeframes	`trade_genius.v5_di_1m_5m()`
First-hour high (L-P1-G4)	`trade_genius.v5_first_hour_high()`
Opening-range-low (S-P1-G4)	`trade_genius.v5_opening_range_low_5m()`
Lock all tracks	`trade_genius.v5_lock_all_tracks(reason)`
Persistence	`paper_state.py` — extends save/load with v5 keys
Tests	`smoke_test.py` (search for `L-P` or `S-P`)

16.4 Migration from v4

A v4 paper-state file (no `v5_*` keys) loads cleanly: the loader treats each missing track as IDLE via `tiger_buffalo_v5.load_track()`. The first session under v5.0.0 starts with empty track dicts; the state machine builds up state lazily as gates pass and stages fire. v4 `paper_trades`, `trade_history`, `positions`, `short_positions`, and the daily entry counters all roll forward unchanged.

16.5 v5.0.x → v5.5.0 — what changed (and what didn't)

The Tiger/Buffalo state machine in §6 is the same code in v5.5.0 as it was in v5.0.0. Every v5.1.x / v5.2.x / v5.3.x / v5.4.x / v5.5.0 release is observation-, infrastructure-, or display-only:

Release	Subject	Behavior change?
v5.0.2	Dockerfile `COPY tiger_buffalo_v5.py` + infra-guard test	No (hotfix)
v5.0.3	Per-executor trade-confirmation DM auto-learn (chat-map)	No (plumbing)
v5.0.4	Revert v5.0.3 paper-key fallback (paper/live key strict)	No (safety)
v5.1.0	Forensic Volume Filter (`volume_profile.py`) — SHADOW	No (logging)
v5.1.1	Env-driven A/B toggles + 3-config parallel shadow logging	No (logging)
v5.1.2	Forensic capture: bar archive + indicator snapshots	No (logging)
v5.1.3	Finnhub data source removed (Yahoo + Alpaca + FMP only)	No (cleanup)
v5.1.4	Equity-aware position sizing (`MAX_PCT_PER_ENTRY` / `MIN_RESERVE_CASH`)	No (sizing safety)
v5.1.5	`/test` command timeout fix	No (UX)
v5.1.6	`BUCKET_FILL_100` 5th shadow config + V510-VEL/IDX/DI streams	No (logging)
v5.1.8	SQLite-backed persistence on `/data/state.db` (replaces JSON)	No (storage)
v5.1.9	`REHUNT_VOL_CONFIRM` + `OOMPH_ALERT` shadow configs (7 active)	No (logging)
v5.2.0	Real-time shadow-strategy P&L tracker on dashboard	No (display)
v5.2.1	Alpaca order idempotency + startup broker reconcile + shadow accounting fixes (H1/H2/H3/M3/M4)	No (safety)
v5.3.0	Shadow strategies moved to dedicated tab + per-config detail	No (display)
v5.3.1	Doc refresh: ARCHITECTURE.md + `trade_genius_algo.pdf` to v5.3.0 state	No (docs)
v5.4.0	Offline backtest CLI (`backtest/` package + `python -m backtest.replay` with `--validate`)	No (offline)
v5.4.1	Shadow tab charts (equity curves + day heatmap + win-rate sparklines) + `/api/shadow_charts` endpoint	No (display)
v5.4.2	Doc refresh: ARCHITECTURE.md + `trade_genius_algo.pdf` to v5.4.1 state	No (docs)
v5.5.0	Nightly backtest scheduler + `/api/backtest_latest` + Shadow tab "Latest backtest" card + `--export-equity` CLI flag	No (offline + display)

Result: a v5.0.0 paper-state file still loads cleanly on v5.5.0 (via the v5.1.8 one-shot JSON-to-SQLite import path); the synthetic harness 50/50 byte-equal goldens still match; every entry, exit, sizing, and stop-placement decision is identical to v5.0.0.

17. Forensic Volume Filter (Anaplan logic) — v5.1.0 → v5.4.1

The §17 layer asks a sharper volume question than the legacy "is volume high" tests: *is this minute's volume higher than the 55-trading-day seasonal average for THIS exact ET timestamp?* Shipped in v5.1.0 in **shadow mode** (computed and logged, but no entry decision changes), extended in v5.1.1 with env-driven A/B toggles and 3-config parallel shadow logging, in v5.1.2 with `GEMINI_A` as a 4th shadow config, in v5.1.6 with `BUCKET_FILL_100` as a 5th plus the bucket-fill velocity streams, and in v5.1.9 with two event-driven extras (`REHUNT_VOL_CONFIRM` and `OOMPH_ALERT`) that own their own virtual positions. **Seven** shadow configs are active through v5.3.0; the gate remains observation only — `VOL_GATE_ENFORCE` defaults to `0` and is unchanged.

17.1 Volume Library — parameters

Parameter	Value
Window	55 NYSE trading days, weekends/holidays excluded.
Bucket	Per-minute, ET, `"0931".."1559"`. 09:30 auction excluded.
Baseline feed	Alpaca historical 1m bars, `feed=sip`, `end < now-16min`.
Live feed	Alpaca websocket `/iex` (free-plan, 30-symbol cap).
Cross-feed scale	Per-ticker IEX/SIP volume ratio over the window. Median is published on the IEX scale.
Per-bucket store	`{"median": int, "p75": int, "p90": int, "n": int}`.
Holiday calendar	Hard-coded NYSE list for 2026-2027 in `volume_profile.py`. No new dep.
Stale threshold	36 hours since `build_ts_utc`.
Disk format	`/data/volume_profiles/<TICKER>.json` (atomic tmp+rename).
Rebuild cadence	Nightly at 21:00 ET (daemon thread); synchronous on startup if any profile is missing or stale.
30-symbol cap	If `len(TICKERS) > 30`, the module hard-disables itself (`VOLUME_PROFILE_ENABLED = False`); the bot trades normally with G4 short-circuited to `DISABLED`.
Websocket reconnect	Jittered backoff; 5-min REST replay on resume to repopulate the in-memory volume table.

17.2 V-P1 grid (legacy fixed-threshold evaluator)

`evaluate_g4(ticker, minute_bucket, current_volume, profile, qqq_current_volume, qqq_profile, stage)` returns `{green, reason, ticker_pct, qqq_pct, rule}` with the original v5.1.0 fixed thresholds (Stage 1: ticker ≥ 120% AND QQQ ≥ 100%; Stage 2: ticker ≥ 100%). Preserved unchanged so v5.1.0 grep tooling and the synthetic harness 50/50 byte-equal still pass.

17.3 Env-driven active config + helper (v5.1.1+)

`load_active_config()` reads six env vars at module-import time:

Env var	Default	Role
`VOL_GATE_ENFORCE`	`0`	Master enforcement flag. Stays `0` through v5.1.2.
`VOL_GATE_TICKER_ENABLED`	`1`	Active config: anchor on per-ticker.
`VOL_GATE_INDEX_ENABLED`	`1`	Active config: anchor on QQQ.
`VOL_GATE_TICKER_PCT`	`70`	Active-config ticker threshold (% of ToD avg).
`VOL_GATE_QQQ_PCT`	`100`	Active-config QQQ threshold (% of ToD avg).
`VOL_GATE_INDEX_SYMBOL`	`QQQ`	Hard-locked to QQQ.

Defaults reflect the **Apr 20-24 backtest recommendation**: 70/100 was the best risk-adjusted config (11 trades, +$482.90, 82% win rate; retains 96% of upside vs unfiltered). Garbage-int env values fall back to defaults rather than crash. Env reads are dict-lookup-cheap; the design intent is "set once at deploy, don't flip mid-week."

`evaluate_g4_config(ticker, minute_bucket, current_volume, profile, index_current_volume, index_profile, *, ticker_enabled, index_enabled, ticker_pct, index_pct)` is the per-anchor configurable evaluator used for the parallel shadow lines. It returns `{verdict, reason, ticker_pct, qqq_pct}` with `verdict ∈ {PASS, BLOCK}` and `reason ∈ {OK, LOW_TICKER, LOW_QQQ, STALE_PROFILE, NO_BARS, NO_PROFILE, DISABLED}`.

17.4 `SHADOW_CONFIGS` — fixed analysis configs (v5.1.1 → v5.1.9)

`volume_profile.SHADOW_CONFIGS` is a hard-coded **5-tuple** (was 3-tuple in v5.1.1; `GEMINI_A` was added in v5.1.2; `BUCKET_FILL_100` was added in v5.1.6 to support the Bucket-Fill Protocol intraminute velocity capture — see §17.4a). Two further configs ship in v5.1.9 as event-driven extras outside the tuple:

Name	`ticker_enabled`	`index_enabled`	`ticker_pct`	`index_pct`	Source
`TICKER+QQQ`	True	True	70	100	`volume_profile`
`TICKER_ONLY`	True	False	70	(unused)	`volume_profile`
`QQQ_ONLY`	False	True	(unused)	100	`volume_profile`
`GEMINI_A`	True	True	**110**	**85**	`volume_profile`
`BUCKET_FILL_100`	True	True	**100**	**100**	`volume_profile`
`REHUNT_VOL_CONFIRM`	n/a	n/a	event-driven (≥100% bucket median + DI≥25 within 10 min after `HARD_EJECT_TIGER`)	—	`_V521_EXTRA_SH` (v5.1.9)
`OOMPH_ALERT`	n/a	n/a	event-driven (DI≥25 + BUCKET_FILL≥100% on minute 1; DI≥25 confirm on minute 2)	—	`_V521_EXTRA_SH` (v5.1.9)

These are NOT env-driven — env vars only choose which one would gate trades if `VOL_GATE_ENFORCE=1`. The point is that every line of shadow data is comparable across all seven configs post-hoc. The **v5.2.1 M3** helper `_v521_all_shadow_config_names()` is the canonical union; close-fanout (`_v520_close_shadow_all`) and EOD code paths iterate it so any new tuple entry plus the event-driven extras is picked up automatically.

17.4a Bucket-Fill Velocity (v5.1.6)

The `BUCKET_FILL_100` row above is paired with three new log streams that capture the data needed to evaluate the Bucket-Fill Protocol post-hoc (see Gene's design note). All three are pure observation; none of them touch the trading decision.

- `[V510-VEL] ticker=X minute=HH:MM second=N running_vol=V bucket=B pct=P qqq_pct=Q` fires **once per (ticker, minute)** on the FIRST tick where running IEX volume crosses 100% of the bucket median for the active candle. This is the data that lets us evaluate the "fires at second 40" velocity hypothesis — the closed-minute backtest cannot see this.
- `[V510-IDX] spy_close=X spy_pdc=Y spy_above=Y/N qqq_close=A qqq_pdc=B qqq_above=Y/N` fires once per candidate consideration. Required for the L-P1 / S-P1 index-direction leg of the protocol (QQQ > PDC AND SPY > PDC, long; mirror for short).
- `[V510-DI] ticker=X di_plus_t-1=A di_plus_t=B di_minus_t-1=C di_minus_t=D double_tap_long=Y/N double_tap_short=Y/N` fires once per candidate consideration. Required for the L-P2 / S-P2 "double-tap" leg (DI+ > 25 on both the prior and current 1-minute bar; mirror DI- for short). DI+ / DI- are computed via Wilder's smoothing in `indicators.py` (`di_plus` , `di_minus`).

17.5 Shadow log lines

Per candidate, `_shadow_log_g4` emits **6** log lines (one back-compat plus five `[CFG=...]` lines for the tuple). The v5.1.9 event-driven extras emit one additional `[V510-SHADOW][CFG=...]` line each on their own trigger conditions:

```
[V510-SHADOW] ticker=AMD bucket=1448 stage=1 g4=GREEN ticker_pct=84 qqq_pct=112 reason=OK entry_decision=ENT
[V510-SHADOW][CFG=TICKER+QQQ][PCT=70/100] ticker=AMD bucket=1448 stage=1 t_pct=84 qqq_pct=112 verdict=PASS
[V510-SHADOW][CFG=TICKER_ONLY][PCT=70]    ticker=AMD bucket=1448 stage=1 t_pct=84              verdict=PASS
[V510-SHADOW][CFG=QQQ_ONLY][PCT=100]      ticker=AMD bucket=1448 stage=1            qqq_pct=112 verdict=PASS
[V510-SHADOW][CFG=GEMINI_A][PCT=110/85]   ticker=AMD bucket=1448 stage=1 t_pct=84 qqq_pct=112 verdict=BLOCK
[V510-SHADOW][CFG=BUCKET_FILL_100][PCT=100/100] ticker=AMD bucket=1448 stage=1 t_pct=84 qqq_pct=112 verdict=
[V510-SHADOW][CFG=REHUNT_VOL_CONFIRM] ticker=AMD side=long bucket=1453 t_pct=104 di_plus=27 verdict=PASS
[V510-SHADOW][CFG=OOMPH_ALERT] ticker=AMD side=long bucket=1448 minute1_di=27 minute1_bucket=104 minute2_di=
```

`entry_decision` always reflects what the bot actually did — these lines never gate the decision. The v5.1.9 extras drop the `entry_decision` field because they fire post-hoc relative to the underlying live trade.

17.6 Failure modes

Condition	Reason
Profile file missing for ticker	`NO_PROFILE`
Profile older than `STALE_HOURS` (36h)	`STALE_PROFILE`
Bucket not in profile (e.g. 09:30 or 16:00)	`NO_BUCKET_<T>_<bucket>`
`len(TICKERS) > 30` (free IEX cap exceeded)	`DISABLED`
Ticker pct < threshold and ticker_enabled	`LOW_TICKER`
QQQ pct < threshold and index_enabled	`LOW_QQQ`

17.7 Rollout plan (current)

1. **v5.1.0** — G4 computed and logged. Existing entry decisions unchanged.
2. **v5.1.1** — Env-driven active config + 3 parallel shadow configs per candidate. Defaults preserve v5.1.0 behavior.
3. **v5.1.2** — `GEMINI_A` added as 4th shadow config; forensic capture (§18) closes the asymmetric blind spot where we only logged what fired. Still SHADOW.
4. **v5.1.6** — `BUCKET_FILL_100` added as 5th shadow config plus the `[V510-VEL]` / `[V510-IDX]` / `[V510-DI]` velocity streams (§17.4a).
5. **v5.1.9** — `REHUNT_VOL_CONFIRM` and `OOMPH_ALERT` added as event-driven extras (now 7 active configs).
6. **v5.2.0** — Real-time shadow P&L tracker (`shadow_pnl.py`) lifts the data from append-only logs into a queryable SQLite table so the dashboard can render running P&L per config.
7. **v5.3.0** — Shadow strategies move to a dedicated dashboard tab with click-to-expand per-config detail (open positions + last 10 closed trades). See §9.5 for the wiring.
8. **v5.4.0** — Offline `backtest/` replay package + `python -m backtest.replay` CLI with `--validate` mode pairing replay vs prod `shadow_positions` (§20).
9. **v5.4.1** — Shadow tab charts (equity curves, day-P&L heatmap, rolling 20-trade win-rate sparklines) backed by `/api/shadow_charts` (§21).
10. **Observation window** — Val collects multiple weeks of 7-config shadow data alongside `[V510-CAND]` / `[V510-MINUTE]` / `[V510-VEL]` / `[V510-IDX]` / `[V510-DI]` streams.
11. **Future PR** — Flip `VOL_GATE_ENFORCE=1` once Val has chosen the best config off live data; G4 joins G1/G2/G3 as a hard ARM precondition (see §19.1).

18. Forensic Capture (v5.1.2)

v5.1.2 closes four data gaps so any future backtest is fully replayable from disk:

1. We only logged trades that **fired** — never candidates that didn't.
2. We only logged at the **active threshold** — not the baseline.
3. We did **not persist 1m bars** to disk.
4. We did **not record indicator state** at decision time.

All four streams are observation-only. None of them touch entry, exit, sizing, or stop placement.

18.1 Bar archive (`bar_archive.py` — Tier-1.1)

For every minute close per ticker (+ QQQ + SPY) in the active TICKERS list, append one JSONL line to `/data/bars/YYYY-MM-DD/{TICKER}.jsonl`. Schema:

```
{
  "ts": "...", "et_bucket": "1448",
  "open": ..., "high": ..., "low": ..., "close": ...,
  "iex_volume": ..., "iex_sip_ratio_used": ...,
  "bid": ..., "ask": ..., "last_trade_price": ...
}
```

Append in `a` mode; on Linux ext4 a write of < `PIPE_BUF` (4096) is atomic, and the lines are ~150 bytes — no tmp+rename needed. Lazy directory creation. **Failure-tolerant — never raises into the trading loop.** Disk projection: ~18 tickers × 390 minutes × ~150 bytes ≈ 1 MB/day. The 30-symbol IEX cap from v5.1.0 still bounds the universe. Stale or empty minutes write nothing. A daemon thread calls `cleanup_old_dirs(retain_days=90)` nightly to delete dated directories older than 90 days.

18.2 Per-minute volume log (`[V510-MINUTE]` — Tier-1.2)

One line per ticker per minute close, regardless of candidate state:

```
[V510-MINUTE] ticker=AMD bucket=1448 t_pct=84 qqq_pct=112 close=346.19 vol=12345
```

This lets us replay "what if the candidate threshold itself were different" without re-pulling 1m bars from Alpaca.

18.3 Skipped-candidate log (`[V510-CAND]` — Tier-1.3)

Pre-v5.1.2 we only logged candidates that fired. v5.1.2 emits one line on **every entry consideration** — fired AND not-fired:

```
[V510-CAND] ticker=AMD bucket=1448 stage=1 fsm_state=ARMED entered=NO reason=NO_BREAKOUT
            t_pct=84 qqq_pct=112 close=346.19 stop=null
            rsi14=null ema9=null ema21=null atr14=null vwap_dist_pct=null spread_bps=null
```

`reason ∈ {NO_BREAKOUT, STAGE_NOT_READY, ALREADY_OPEN, COOL_DOWN, MAX_POSITIONS, BREAKOUT_CONFIRMED}`. Wired into the entry-consideration loop next to the existing `_shadow_log_g4` call. All `indicators.*` values render as `null` (not `0.0`) on insufficient bars.

18.4 Entry log enrichment (`[V510-ENTRY]` — Tier-1.4)

When a trade fires, a new line is emitted alongside the existing entry surface (Telegram + paper_log) carrying bid/ask + account state:

```
[V510-ENTRY] ticker=AMD bid=346.18 ask=346.20
             cash=85432.10 equity=99214.55
             open_positions=2 total_exposure_pct=18.4 current_drawdown_pct=0.0
```

Strictly additive — the existing entry log line, paper_log entry, and Telegram card are unchanged byte-for-byte (the synthetic harness 50/50 byte-equal goldens still pass).

18.5 FSM transition log (`[V510-FSM]` — Tier-2.1)

Pure observation hook. Refuses to emit on `from == to` no-ops (asserted by smoke test). Format:

```
[V510-FSM] ticker=AMD from=IDLE to=WATCHING reason=VOL_SPIKE_DETECTED bucket=1445
```

v5.1.2 ships the emitter only; the wider FSM-call-site sweep is intentionally minimal so we do not accidentally change v5.0.0 Tiger/Buffalo behavior. A future PR will fan the emitter out to every transition site.

18.6 Pre-trade indicator snapshots (`indicators.py` — Tier-2.2)

Pure functions, no `trade_genius` imports, no exceptions raised on bad input:

Function	Inputs	Output	None when
`rsi14(closes)`	closes (newest last)	float	< 15 closes
`ema9(closes)`	closes	float	< 9 closes
`ema21(closes)`	closes	float	< 21 closes
`atr14(bars)`	bars with high/low/close	float	< 15 bars
`vwap_dist_pct(bars)`	bars with H/L/C/volume	float	no bars or zero volume
`spread_bps(bid, ask)`	bid, ask	float	missing or non-positive

All values are wired into `[V510-CAND]` so every candidate moment carries the indicator state at decision time. Callers MUST emit `null` (not `0.0`) into log lines on `None` returns.

18.7 Out of scope (deferred)

News / halt flags (needs Polygon or Benzinga subscription); L2 / order-book snapshots; tick-level trades; `VOL_GATE_ENFORCE=1` ; new env-driven configs beyond v5.1.1; adaptive runtime config switching.

19. Long permission gates (G1..G4)

19.1 Four-light gate set (v5.1.0+)

Gate	Light name	Source	Status through v5.4.1
G1	Index Alpha	Tiger/Buffalo §6.2	enforced
G2	Ticker Alpha	Tiger/Buffalo §6.2	enforced
G3	5m OR Structure	Tiger/Buffalo §6.2	enforced
G4	Volume	`volume_profile.evaluate_g4(_config)`	**shadow only — `VOL_GATE_ENFORCE=0` through v5.4.1; flips in a future PR**

Through v5.4.1 the bot logs G4 every minute via `_shadow_log_g4` but does not block any entry on it. The short-side gates read the same baseline; no short-specific code change.

---

20. Offline backtest CLI (v5.4.0)

The `backtest/` package is a self-contained offline replayer. It reads the JSONL bar archive written by `bar_archive.write_bar` and replays one (or all) of the `volume_profile.SHADOW_CONFIGS` gate rules over a trading-day range, producing a CSV ledger of entry/exit pairs and an optional replay-vs-prod validation report.

20.1 How to run

```
python -m backtest.replay \
    --start 2026-04-20 --end 2026-04-24 \
    --config GEMINI_A \
    [--validate] \
    [--out ./backtest_out/] \
    [--bars-dir /data/bars/] \
    [--state-db /data/state.db]
```

`--config` accepts any `SHADOW_CONFIGS` name (`TICKER+QQQ` , `TICKER_ONLY` , `QQQ_ONLY` , `GEMINI_A` , `BUCKET_FILL_100`) or the literal `ALL` to fan out across every config in one run.

20.2 What `--validate` does

In validate mode, after the replay completes, the CLI queries `shadow_positions` in `state.db` for the same `config_name` over the same date range and pairs each predicted replay entry to a prod row by ticker + side + entry timestamp within ±60s. The output report `<out>/<config>_<start>_<end>_validation.md` lists:

- **match rate** = matches / total prod entries
- **REPLAY_ONLY** entries (potential false-positive — replay would have entered, prod did not)

- **PROD_ONLY** entries (potential false-negative — prod entered, replay did not)
- average entry-price and exit-price drift across matched pairs

Exit code is 0 when match rate ≥ 0.95, else 1, so CI can gate on it.

20.3 CSV ledger columns

The ledger CSV at `<out>/<config>_<start>_<end>.csv` carries one row per closed entry/exit pair plus a leading `# summary:` comment line.

column	meaning
`ticker`	uppercase symbol
`side`	`BUY` (long) or `SHORT`
`entry_ts`	UTC ISO timestamp the position opened on a bar close
`entry_price`	bar-close price at entry
`exit_ts`	UTC ISO timestamp the position closed
`exit_price`	bar-close price at exit
`qty`	shares (sized off a $1000-per-position budget)
`pnl_dollars`	realized P&L in $; direction-aware
`pnl_pct`	realized P&L as % of entry price
`exit_reason`	`trail_stop` · `hard_eject` · `eod`

The `# summary:` line gives quick totals: trade count, wins, losses, total P&L, win rate.

20.4 Reused logic

Gate-pass evaluation (`_gate_pass`) and the entry → exit pairing follow the same model proven out in `backtest_v510/replay_gate.py` and `backtest_v510/replay_gene_configs.py` — the `pnl_per_pair` math is ported directly. The MVP intentionally uses bar-close prices and a simple trail/eject/eod policy rather than re-implementing every nuance of `trade_genius.py`'s live decision tree; the engine's job is *deterministic, diff-able replay*, and `--validate` is the canary that flags drift between replay and prod.

---

21. Shadow tab charts (v5.4.1)

v5.4.1 layers three Chart.js visualizations on the Shadow tab without introducing any new SQLite tables — every chart is derived from the existing v5.2.0 `shadow_positions` rows (closed trades only, `exit_ts_utc IS NOT NULL`) by a new HTTP endpoint.

21.1 Endpoint — `GET /api/shadow_charts`

Defined in `dashboard_server.py`. Same session-cookie auth as the rest of `/api/*`. Server-side cache: a single lock-protected `(ts, payload)` tuple with a 30 s TTL — the same pattern used by `/api/indices` — so multiple browsers polling the Shadow tab in parallel collapse to one SQLite read per window. The response always emits all 7 configs in a fixed order; configs with no closed trades render as empty arrays rather than missing keys.

Response shape:

```
{
  "configs": {
    "GEMINI_A": {
      "equity_curve":     [{"ts": "...", "cum_pnl": 0.0}, ...],
      "daily_pnl":        [{"day": "YYYY-MM-DD", "pnl": 0.0, "trades": 0}, ...],
      "win_rate_rolling": [{"ts": "...", "win_rate": 0.0}, ...]
    },
    "...": { ... }
  },
  "as_of": "<UTC ISO ts>"
}
```

- `equity_curve` — cumulative realized P&L sampled per closed trade.
- `daily_pnl` — per-trading-day total P&L plus closed-trade count.
- `win_rate_rolling` — rolling 20-trade window win rate; the array is empty (and the chart hidden client-side) for any config with fewer than 20 closed trades.

21.2 Frontend — three chart groups on the Shadow tab

Chart.js 4.4.0 is loaded from jsDelivr CDN with a `defer` attribute. The chart code falls back gracefully if `window.Chart` is undefined — empty wrappers render and the rest of the dashboard keeps working.

Three vertically-stacked groups sit above the existing per-config rows:

1. **Equity curves** — one Chart.js line chart per config (~100 px desktop, ~80 px mobile). Y-axis cumulative $, X-axis time.
2. **Day-P&L heatmap** — single ~300 px scatter chart, rows = configs, columns = trading days, cell color = green/red intensity scaled to abs-max P&L across all cells.
3. **Rolling win-rate sparklines** — one per config (~60 px), Y-axis 0–1, hidden when the config has < 20 closed trades.

Each config gets a stable hue across all three groups (`SHADOW_CFG_COLORS`) so `GEMINI_A`'s equity curve, heatmap row, and win-rate sparkline are always the same color. Axis colors and gridlines read from existing CSS variables (`--text-dim`, `--border`); no new color literals.

The "Charts" header is collapsible (click / Enter / Space toggles). Default is **expanded on desktop and collapsed on ≤ 720 px viewports** so the Shadow tab is not dominated by chart real estate on a phone.

21.3 Polling

Tab-aware: `/api/shadow_charts` is fetched once on Shadow-tab activation and then every 60 s **only while the Shadow tab is active** — Main / Val / Gene ticks skip the call entirely. Matches the existing `pollExecutor` pattern.

---

22. Nightly backtest (v5.5.0)

v5.5.0 wires the v5.4.0 offline replayer into the standard scheduler so a full replay-vs-prod check runs automatically every night against the prior trading day. The result is surfaced through a new dashboard endpoint and a "Latest backtest" card at the top of the Shadow tab. The CLI also gains an `--export-equity` flag that produces dashboard-shape chart JSON offline from any backtest run.

22.1 Scheduler entry

A new job is registered in `scheduler_thread()` inside `trade_genius.py`:

```
("everyday", "22:00",
 lambda: threading.Thread(target=_run_nightly_backtest,
                          daemon=True).start()),
```

22:00 ET is post-EOD reconciliation (15:55 ET `eod_close`, 15:58 ET `send_eod_report`) and aligns with ~02:00 UTC year-round (within one hour during DST shifts). The wrapper `_run_nightly_backtest` simply calls `scripts.nightly_backtest.run_nightly()` on a daemon thread and logs the per-config count + as-of date returned by the run. **Railway cron is intentionally not used** — the in-process scheduler is simpler to reason about, shares the existing `persistence.was_fired` / `mark_fired` idempotency, and keeps the failure mode local (a container restart only loses the in-flight run, never restarts the container itself).

`scripts/nightly_backtest.py`:

1. Picks the most recent **completed** NYSE trading day via `volume_profile.is_trading_day` (today is excluded — the nightly runs after market close, so "completed" is the prior session).
2. Iterates every entry in `volume_profile.SHADOW_CONFIGS`, replays that day with `backtest.replay.replay_one_day`, writes the per-config CSV ledger to `/data/backtest_reports/<CFG>_<DAY>_<DAY>.csv`, and (when validation is enabled) also writes `<CFG>_<DAY>_<DAY>_validation.md` via `backtest.replay.write_validation_report`.
3. Captures all stdout/stderr to `/data/backtest_reports/<DAY>_log.txt` (via `redirect_stdout` / `redirect_stderr` of an in-memory `io.StringIO`).
4. Calls `backtest.replay.write_equity_export` to write `/data/backtest_reports/<DAY>_charts.json` (same shape as `/api/shadow_charts`).
5. Writes a top-level `/data/backtest_reports/latest.json` index keyed by config name with: `match_rate`, `replay_only_count`, `prod_only_count`, `total_pnl`, `trade_count`, `win_rate`, `exit_code`. Top-level keys: `as_of`, `generated_at`, `configs`.
6. **Always exits 0** so Railway never restarts the container on validation drift. Drift is surfaced through the dashboard, not the orchestrator.

The directory root and bar/state-DB paths are overridable via `BACKTEST_REPORTS_DIR`, `BACKTEST_BARS_DIR`, `BACKTEST_STATE_DB` env vars (defaults: `/data/backtest_reports`, `/data/bars`, `/data/state.db`).

22.2 Endpoint — `GET /api/backtest_latest`

`dashboard_server.h_backtest_latest` reads the file at `<BACKTEST_REPORTS_DIR>/latest.json` and returns its contents verbatim. When the file is missing or unparseable it returns a graceful placeholder so the dashboard never has to special-case 5xx:

```
{ "status": "no_data", "as_of": null, "configs": {} }
```

When the file exists, the response is the raw `latest.json` payload (`status` defaults to `"ok"` if not present in the file). Same session-cookie auth as the rest of `/api/*`. No server-side cache — the file is small and only rewritten once per day.

22.3 Frontend — "Latest backtest" card

The Shadow tab gains a new collapsible card, positioned **above** the v5.4.1 Charts subsection. The card head shows the as-of date and a summary chip whose background color reflects the aggregate match-rate band (green ≥0.95, yellow 0.90–0.95, red <0.90). The body has two parts:

1. **Headline** — three lines: aggregate match rate (color-coded), best config by total P&L (name + $ + win rate), worst config by total P&L.
2. **Per-config table** — one row per `SHADOW_CONFIGS` entry with columns: Config, Match, Replay-only, Prod-only, Total P&L, Trades, Win rate. Match-rate cells are color-coded with the same band logic as the headline.

Default state is **expanded on desktop and collapsed on ≤ 720 px viewports**, mirroring the Charts card. Polling is tab-aware: the card fetches `/api/backtest_latest` immediately on Shadow-tab activation and then every 60 s only while the Shadow tab is active. Implemented as a self-contained module (`__tgPollBacktestLatest` / `_btlRender`) — it does not touch the v5.4.1 Chart.js click/tooltip handlers.

22.4 CLI — `--export-equity`

`python -m backtest.replay` accepts a new optional flag:

```
python -m backtest.replay \
  --start 2026-04-22 --end 2026-04-22 \
  --config ALL --validate \
  --export-equity ./out/2026-04-22_charts.json \
  --out ./out/
```

When set, after all per-config CSV ledgers and optional validation reports are written, the replayer also writes a JSON file at the given path. The shape mirrors `GET /api/shadow_charts`:

```
{
  "configs": {
    "TICKER+QQQ": {
      "equity_curve":     [{"ts": "...", "cum_pnl": 0.0}, ...],
      "daily_pnl":        [{"date": "YYYY-MM-DD", "pnl": 0.0,
                            "trades": 0}, ...],
      "win_rate_rolling": [{"trade_idx": 20, "win_rate": 0.55}, ...]
    },
    "...": { ... }
  },
  "as_of": "<UTC ISO ts>"
}
```

Computation matches `dashboard_server._shadow_charts_payload`: cumulative P&L per closed trade (sorted by `exit_ts`), daily P&L bucketed on the `exit_ts` date, rolling 20-trade win rate with the same warm-up gate. The nightly scheduler entry uses this helper to write `<DAY>_charts.json`; a developer can also invoke it directly from any one-off backtest run to render the same charts offline.

Last refresh: April 2026, against `BOT_VERSION = "5.5.0"`.